

Logyx — Documento di Design

Sep 28, 2026 · @Daniele

Visione e obiettivo

Un solo linguaggio per fare ogni cosa: la versatilità e la concisione di Python, la leggerezza e la comodità web di PHP, ma compilato a codice macchina e capace di girare sia sul server sia sul client.

L'obiettivo è cancellare la divisione tra "linguaggio server" e "linguaggio client". Oggi per il web servono due linguaggi diversi; qui un solo sorgente, con una sola sintassi, gira nativo sul server e come WebAssembly nel browser, con deploy semplice come PHP e velocità vicina al C.

Principi guida

Sono le regole non negoziabili che decidono i dubbi di design futuri. Quando qualcosa entra in conflitto, si torna qui.

1. **Un sorgente, molti mondi.** Lo stesso codice gira nativo sul server e come WASM nel client, senza duplicazione e senza un secondo linguaggio.
2. **Facile per default, veloce a richiesta.** Si scrive dinamico e conciso come PHP/Python; si aggiungono i tipi solo dove serve prestazione (gradual typing).
3. **Sintassi esplicita, mai posizionale.** Delimitatori chiari, indentazione solo cosmetica, punto e virgola opzionale.
4. **Batterie da web incluse.** Routing, template embedding, HTTP e database nella libreria standard, alla PHP; deploy semplice come obiettivo di prima classe.
5. **Interoperabilità come fondamento.** Accesso a C (lingua franca) e alle crates Rust dal primo giorno.
6. **Il linguaggio resta piccolo, l'ecosistema copre i mondi.** Il cuore del linguaggio sta in testa a una persona; la copertura arriva dalle librerie, non dal linguaggio che si gonfia.

Domanda aperta: se due principi confliggono, quale vince? Proposta iniziale: la sicurezza (principio implicito nel confine server/client) batte tutto, poi la semplicità (2, 3), poi la portabilità (1).

Nome del linguaggio

Nome scelto: **Logyx**. Verifica fatta sul web (settembre 2026): quasi tutti i nomi brevi ed evocativi sono già usati da altri linguaggi. Logyx conserva la radice λόγος (parola, ragione, linguaggio) e il suono `-yx` di Onyx, ed è risultato libero come nome di linguaggio.

Nome	Verifica sul web	Estensione	Esito
Logyx	Libero: nessun linguaggio omonimo; radice λόγος + suono <code>-yx</code>	<code>.logyx</code> / <code>.lgx</code>	Scelto
Obsyd	Libero (esiste solo una API di dati elettrici, altro dominio)	<code>.obsyd</code>	Alternativa libera
Onyx	Preso: esiste già un linguaggio Onyx che compila a WASM	—	Scartato
Weave / Weft / Nexa / Basalt / Nyxa / Ryft / Kryon / Vyre	Presi da altri linguaggi/progetti	—	Scartati
Logos / λόγος	Preso: Logo educativo e <code>logos</code> , generatore di lexer in Rust	—	Scartato

Una volta scelto il nome, tutto il resto (cartella di progetto, estensione, titolo) prende quel nome.

Sintassi

Non posizionale: la struttura la danno i delimitatori, non l'indentazione. I rientri restano solo per leggibilità e il punto e virgola è opzionale (inserito automaticamente dal compilatore).

Le porzioni che devono girare nel browser si marcano con blocchi espliciti `@start-client ... @end-client`: fuori da essi è tutto server, così si vede a colpo d'occhio dove passa il confine. Un decoratore `@server` / `@client` è la variante breve per un'intera funzione.

```

route "/todo":                                # server (come PHP)
  lista = db.query("SELECT * FROM todo")

  render <html>
    <ul>{ for t in lista: <li>{t.testo}</li> }</ul>

    @start-client
      button.onClick = () => { conta += 1 }    # diventa WASM
    @end-client
  </html>

# con tipi, dove serve prestazione
def somma(a: int, b: int) -> int:
  return a + b

```

Sistema di tipi ed esecuzione

Gradual typing: il codice è dinamico e conciso per default; le annotazioni di tipo sono opzionali e servono a chi vuole controllo e prestazione. Il compilatore usa la type inference per dedurre i tipi dove non sono scritti.

L'esecuzione ha due modalità:

- **Interprete** per lo sviluppo rapido: il linguaggio "gira" subito, sensazione Python.
- **Compilazione a codice macchina** per la produzione, ottenuta transpilando verso Rust e lasciando che `rustc` faccia il lavoro pesante (ottimizzazione, sicurezza di memoria, linking, generazione WASM).

La scelta di transpilare a Rust è strategica: regala quasi gratis server nativo, client WASM e i target desktop/mobile, perché sono tutti bersagli che la toolchain Rust già raggiunge.

Confine server/client

Server e client sono due macchine diverse che non condividono memoria e si parlano solo via rete. Il linguaggio non finge che il confine non esista: lo rende comodo da attraversare senza nascondere i pericoli.

Tre regole inviolabili (leggi fisiche, non scelte di design):

1. **Solo dati serializzabili attraversano.** Numeri, stringhe, liste, oggetti semplici sì; connessioni al DB, handle di file, funzioni no. Il compilatore lo verifica e rifiuta ciò che non può viaggiare.
2. **Il client non è mai fidato.** Nessun segreto nel codice client; ogni dato che arriva dal browser va validato di nuovo sul server. Il confine è anche un confine di sicurezza.

3. **Attraversare costa.** Ogni salto è un viaggio di rete: il design lo rende visibile (i blocchi `@start-client`).

Modello di default scelto: **server-driven** (stile LiveView). Lo stato e la logica vivono sul server, il client resta quasi vuoto, il server rimanda solo le differenze del DOM. Massima leggerezza stile PHP e sicurezza quasi gratis. Per interazioni istantanee si offre come seconda marcia il modello RPC/idratazione.

Modello	Come funziona	Quando
Server-driven (default)	Stato sul server, il client riceve i diff del DOM	Gran parte delle app; massima semplicità e sicurezza
RPC trasparente	Il client chiama funzioni <code>@server</code> come locali; sotto c'è la rete	App reattive, uso offline
Idratazione (SSR)	Il server genera l'HTML, il WASM lo anima	Siti di contenuto, SEO, caricamento veloce

Multi-target

Un solo nucleo, molti bersagli di compilazione. Poiché si transpila a Rust, si ereditano tutti i target che Rust già raggiunge. Il programmatore impara una cosa sola e la porta ovunque.

Bersaglio	Cosa ci fai	Come ci arrivi
Server nativo (Linux x86/ARM)	Backend web, servizi, API	Rust → nativo
Browser (WASM)	Frontend web	Rust → WASM
Desktop (Win/Mac/Linux)	App con finestre	Rust → nativo
Mobile (Android/iOS)	App con touch e sensori	Rust → ARM

Distinzione da incidere nella pietra:

- **Logica portabile** — calcoli, algoritmi, tipi, regole di business: gira identica ovunque. È il grosso di un programma.
- **Capacità di piattaforma** — DOM nel browser, finestre e filesystem sul desktop, GPS e fotocamera sul mobile: diverse per forza, esposte da moduli specifici.

Sul web il "client" è il browser, dove non si può installare nulla: da qui l'obbligo di WASM. Su app desktop/mobile, invece, il runtime nativo può stare su entrambi i lati (modello "installato di qua e di là").

Ecosistema e interoperabilità

Decisione presa: agganciarsi a due ecosistemi nativi, senza mai tradire la compilazione a macchina.

- **FFI con C — la base universale.** Il C ha una ABI stabile: è la lingua franca con cui tutti i linguaggi si parlano. Basta dichiarare la firma e il linker fa il resto. Ogni libreria di sistema è raggiungibile così.
- **Crates Rust — il cavallo di battaglia.** Rust non ha una ABI stabile, quindi le crates si usano solo dall'interno del mondo Rust: questo è il motivo tecnico per cui il backend transpila a Rust invece di puntare direttamente a LLVM.

```
extern "C" {
    fn sqrt(x: float) -> float
}
```

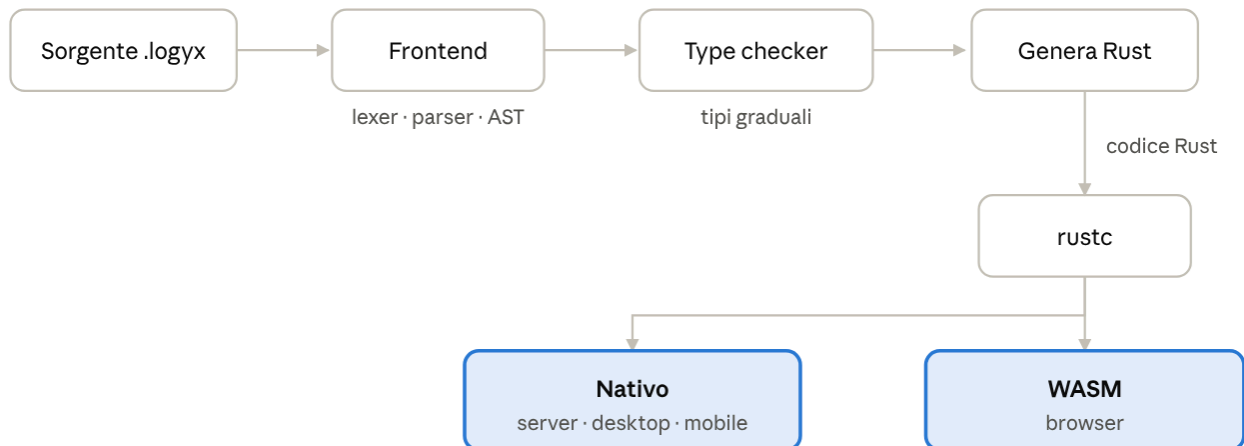
Nota onesta: "compilato nativo puro" e "importo qualsiasi libreria Python al volo" sono in conflitto tecnico (le librerie Python girano dentro l'interprete Python). Qui vince il nativo; l'ecosistema Python resta un'opzione futura via ponte, non un fondamento.

Architettura del compilatore

Non un compilatore monolitico, ma un frontend unico con più backend. Il consiglio pratico: **prototipo in Python o TypeScript** (lexer, parser, interprete: si scrivono programmi veri in pochi giorni), poi **compilatore vero in Rust** per il controllo della memoria e il pattern matching sugli AST.

La pipeline: il sorgente attraversa lexer, parser e type checker; il backend genera codice Rust, e `rustc` lo trasforma nei bersagli finali (nativo per server/desktop/mobile, WASM per il browser).

Un sorgente, un compilatore, molti bersagli



pipeline del compilatore · dal sorgente ai bersagli nativo e WASM

Il frontend è tuo; da `rustc` in poi il lavoro pesante è già fatto, e i bersagli nativo e WASM arrivano dalla stessa toolchain.

Decisioni prese e domande aperte

Tema	Decisione
Ecosistema	FFI con C + interop con le crates Rust
Backend	Transpiling verso Rust (poi <code>rustc</code>), non LLVM diretto all'inizio
Host del compilatore	Prototipo in Python/TypeScript, compilatore vero in Rust
Sintassi	Non posizionale, delimitatori espliciti, <code>;</code> opzionale
Isole client	Blocchi <code>@start-client</code> / <code>@end-client</code> ; default server
Confine server/client	Server-driven di default; RPC/idratazione come seconda marcia
Target	Server nativo, browser WASM, desktop, mobile
Tipi	Gradual typing
Autore	Daniele Deplano — intestazione su ogni sorgente, nessuna firma di terzi

Tema	Decisione
Licenza	AGPL-3.0

Domande ancora aperte:

- Il **nome** del linguaggio (vedi sezione dedicata).
- La **gerarchia dei principi** quando confliggono.
- La grammatica precisa: parole chiave, blocchi (`{ }` o `end`), regole del `;` automatico.
- Come viaggiano i dati oltre il confine: formato di serializzazione e trasporto (WebSocket per il modello server-driven).

Grammatica v0 e ciao mondo multi-target

Prima bozza della grammatica (spec completa in `GRAMMAR.md` nel repo). Scelte v0:

- Blocchi con graffe `{ }` (sintassi non posizionale); `;` opzionale; indentazione solo cosmetica.
- Commenti `//` e `/* */`; logici `and` `or` `not`; `true` / `false` / `nil`.
- Stringhe con interpolazione: `"Ciao {nome}"`. Funzioni con `fn`, tipi opzionali (gradual typing).
- Web: `route` (server), `render` con template HTML, isole client fra `@start-client` e `@end-client`.

Ciao mondo multi-target — un solo file per server e client:

```

route "/" {
  saluto = "Ciao, mondo!" // SERVER (nativo)

  render <html>
    <body>
      <h1>{saluto}</h1>
      <button id="btn">Cliccami</button>
      <p id="out">--</p>

      @start-client // CLIENT (WASM)
        clic = 0
        on "click" of "#btn" {
          clic = clic + 1
          set text of "#out" to "Premuto {clic} volte"
        }
      @end-client
    </body>
  </html>
}

```

Il calcolo di `saluto` e la route girano sul server (nativo); il blocco client viene compilato a WASM. Nessun secondo linguaggio, nessun secondo file.

Stato del prototipo (Python). Nel repo `logyx/prototype/` c'è un interprete funzionante (lexer + parser + interprete tree-walking, senza dipendenze). Esegue il nucleo del linguaggio e il **render lato server**: interpolazione `{expr}` con escaping automatico e blocchi di controllo `{ for x in xs { ... } }` e `{ if cond { ... } }` nel template. Le isole `@start-client` sono rese come segnaposto (diventeranno WASM). Comandi: `render` (stampa l'HTML di una route) e `serve` (server HTTP).

Prossimi passi

1. **Fatto** — prototipo del frontend in Python: nucleo del linguaggio + render lato server (route, `render`, interpolazione con escaping, blocchi `{for}` / `{if}`).
2. **Lato client vero**: isole `@start-client` compilate a WASM.
3. **Moduli/import** e gestione errori nel linguaggio.
4. **Formato del confine**: serializzazione e trasporto server/client (WebSocket per il server-driven).
5. **Compilatore vero in Rust**: transpiling verso Rust, poi `rustc` per nativo e WASM.